

Notes

PLAYLIST 2 NOTES

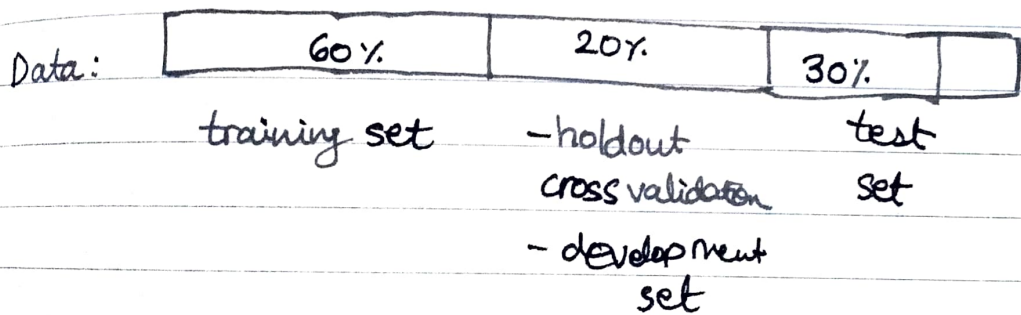
Vid 1: Train / Dev / Test Sets.

ML is a very iterative process.

0 to ∞ $\rightarrow$

#layers, #hidden units, learning rates, activation functions ...

ML has many applications such as CV, NLP, speech, etc.



This was considered good. earlier (for smaller data)

for big data: 98% / 1% / 1%

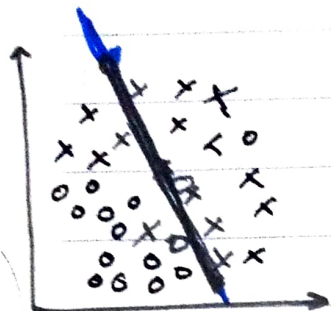
eg. Training set:
Cat pictures from
webpages

Dev / test sets:
Cat pictures from users
using your ~~cat~~ app.

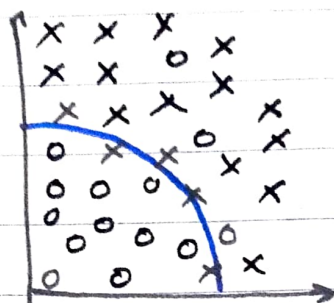
* Make sure dev, test come from same distribution

Notes

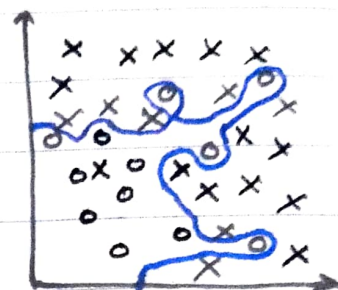
Vid 2: Bias / Variance



High Bias
(underfitting)



"just right"



high Variance
(overfitting)

cat classification



Train set error : 1%

15%

15%

0.5%

Dev set error : 11%

16%

30%

1%

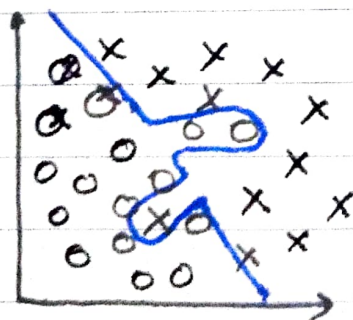
High Var

high bias

high bias
high var.

low bias
low variance

By looking at these errors, we can diagnose that our algorithm has high variance.



"High Bias"
X "High Variance"



Monday

MARCH

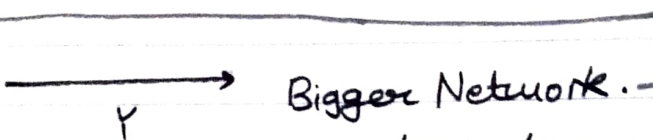
Mo	Tu	We	Th	Fr	Sa	Su	Mo	Tu	We	Th	Fr	Sa	Su
1	2	3	4	5	6	7	8	9	10	11	12	13	14
15	16	17	18	19	20	21	22	23	24	25	26	27	28
29	30	31											

8 am

Vid3: Basic Recipe for ML.

9

High Bias ?
(training data problem)

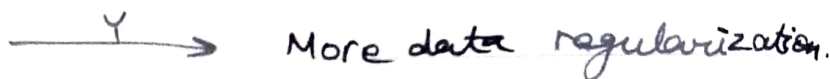


10

11

12

High Variance
(dev set problem)



1 pm

2

↓ N
DONE

3

Vid4: Regularization.

4

Logistic Regression min $J(w, b)$

5

6

7

$$J(w, b) = \frac{1}{m} \sum_{i=1}^m \mathcal{L}(\hat{y}^{(i)}, y^{(i)}) + \frac{\lambda}{2m} \|w\|^2$$

REGULARIZATION
PARAMETER

$$\|w\|^2 = \sum_{j=1}^{n_x} w_j^2 = w^T w. \quad \} \text{ L2 Regularization}$$

8 am
9 There is also a $\frac{\lambda}{2m} b^2$ term, which is usually omitted

10 since it does not make much difference

11 L_1 regularization: 

12
$$\frac{\lambda}{2m} \sum_{i=1}^{n_x} |w_i| = \frac{\lambda}{2m} \|w\|_1$$

1 pm

2 If L_1 is used, w will become "sparse".

3 (W vector will have a lot of zeroes in it)

4 λ is called Regularization parameters.
(Hyperparameter)

5 lambda is a reserved keyword, so we will use
"lambd" for the regularization parameter.

6

In an NN,

9

$$J(w^{[1]}, b^{[1]}; w^{[2]}, b^{[2]}; \dots; w^{[L]}, b^{[L]})$$

10

$$= \frac{1}{m} \sum_{i=1}^m \mathcal{L}(\hat{y}^{(i)}, y^{(i)}) + \frac{\lambda}{2m} \sum_{l=1}^L \|w^{[l]}\|^2$$

11

$$\|w^{[l]}\|_F^2 = \sum_{i=1}^n \sum_{j=1}^{n^{[l-1]}} (W_{ij}^{[l]})^2$$

12

1 pm

$$\dim(w) = (n^{[l-1]} \times n^{[l]})$$

2

This type of norm is called "Frobenius Norm".
denoted by F in subscript.

4

$$dw^{[l]} = \frac{1}{m} \times dz^{[l]} A^{[l-1]T} + \frac{\lambda}{m} w^{[l]}$$

5

$$w^{[l]} : w^{[l]} - \alpha dw^{[l]}$$

6

$$: w^{[l]} - \alpha \left[\frac{1}{m} dz^{[l]} A^{[l-1]T} + \frac{\lambda}{m} w^{[l]} \right]$$

7

$$: w^{[l]} - \frac{\alpha \lambda}{m} w^{[l]} = \alpha (\text{backprop})$$



$\therefore$ L2 Norm regularization is called "weight decay".

Vid 5: Why Regularization reduces overfitting?

9

10 Regularization adds another term:

$$\frac{\lambda}{2m} \sum_{d=1}^D \|w^{[d]}\|_F^2 \quad (\text{this penalizes the weights being too large})$$

11

12 If $\lambda \uparrow \uparrow$, they'll be incentivized to set weight matrix $W \approx 0$.

1 pm The model will then become smaller.

This reduces the variance and brings it closer to the high bias case.

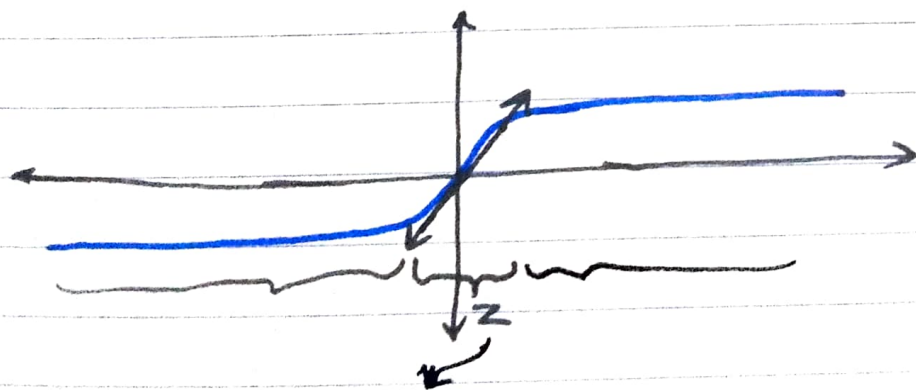
2

3 let's see the activation f^h $\tanh(z)$:

4

5

6



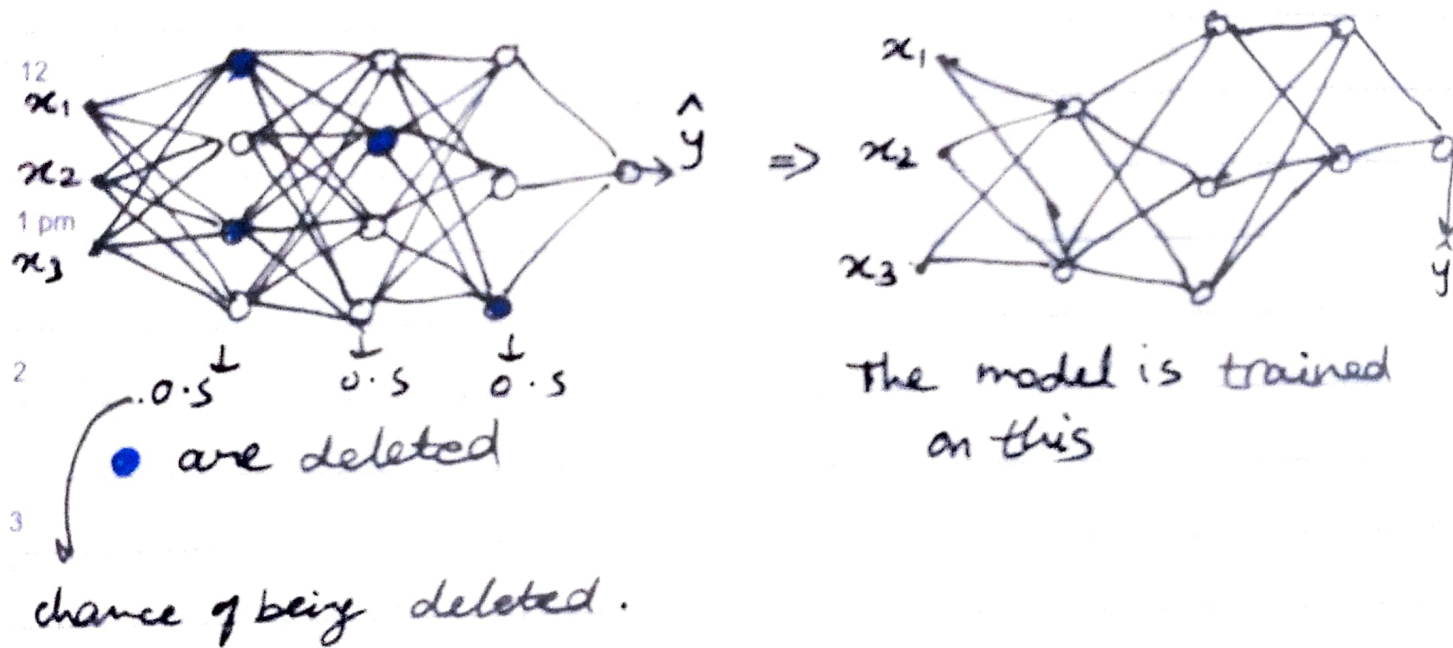
if z is small,

7 if $\lambda \uparrow$, $w^{[d]} \downarrow \therefore z^{[d]} \downarrow$ then $\tanh(z)$ is roughly linear (see graph).

and we have already seen that if $g(z)$ is linear, then entire NN is linear (to be avoided)

Vid 6: Dropout Regularization

- we will make a copy of the NN and we will see if there is a probability of eliminating a node.



Implementing Dropout.

we will illustrate this with $l=3$

8 $d3 = np.random.rand(a3.shape[0], a3.shape[1])$
 9 $< keep_prob.$

10 $keep_prob = 0.8$ // This is the probability of keeping
 11 a node.

12 currently theres a 0.2 chance that a node is deleted.
 (20%)

1 pm $a3 = np.multiply(a3, d3)$ or $a3 *= d3$

2 for every element of $d3$ that is equal to 0, this
 3 ends up zeroing out the corresponding element in $a3$.

4 $a3 /= keep_prob$

lets say a layer has 50 nodes.

5 due to $keep_prob = 0.8$, on avg, 10 nodes will be
 6 deleted.

07 Sunday

7
$$z^{[4]} = W^{[4]} \underline{a^{[3]}} + b^{[4]}$$

reduced
by 20%.

so you ~~multiply~~ ^{divide} it back by $keep_prob$ to $\uparrow$.



Making predictions at test time.

9

$$a^{[0]} = X$$

10

No dropout. (don't want o/p to be random)

11

$$z^{[1]} = w^{[1]} a^{[0]} + b^{[1]}$$

$$a^{[1]} = g^{[0]}(z^{[1]})$$

12

$$z^{[2]} = w^{[2]} a^{[1]} + b^{[2]}$$

$$a^{[2]} = \dots$$

1 pm

⋮
↑
 $\hat{y}$

2

• Vid 7. Understanding drop-out.

3

any node cannot rely on any one feature, because they may get deleted.

4

So we "spread out" the weight.

5

We can also change keep-prob for each layer.

7

(layers which has more hidden units (nodes) may have v low keep-prob relatively.



8 am If you don't want to remove a node, simply set keep-prob to 1.

9

If some layers are more prone to overfitting than others, set keep-prob $\downarrow$.

11 One downside is that if you want to check the performance of gradient descent, it's hard to make sure that your cost f^n is decreasing monotonically.

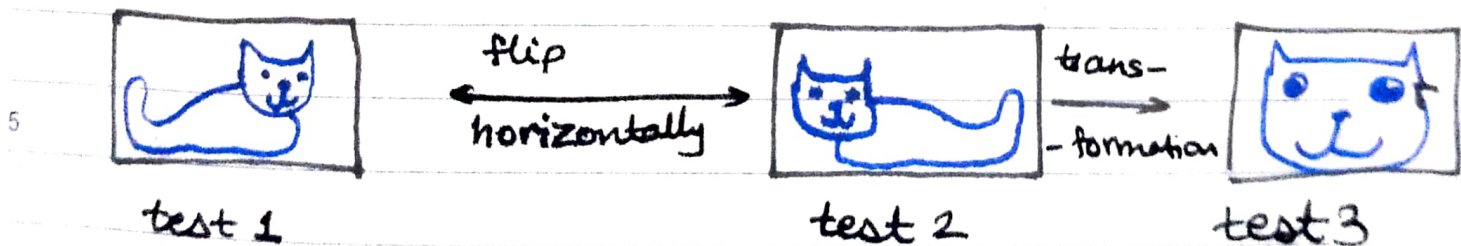
1 pm Sol: Set keep-prob = 1

2 Vid 8: Other regularization methods.

3 eg. we are fitting a cat classifier.

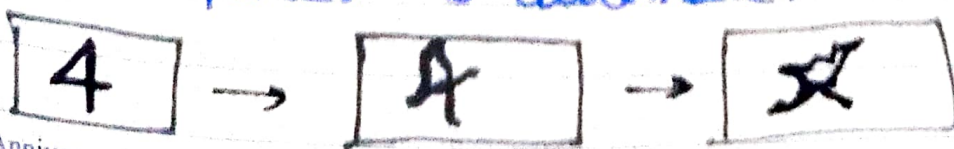
"Data Augmentation:"

4

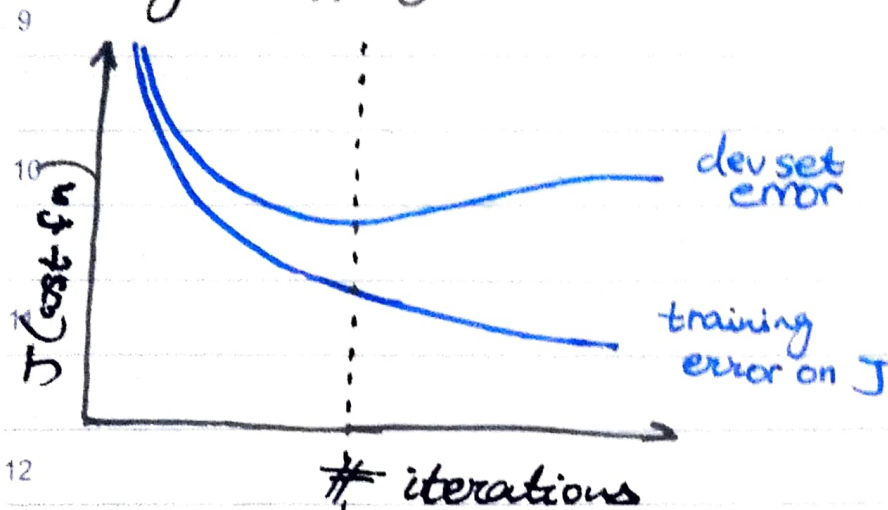


6 $\therefore$ gathering data can be expensive/impossible.
So the test 2 image can also be added to the sample.
2 you can also zoom or slightly rotate to get more images.
This enables us to double the size of training set.

- use opencv to automate.



"Early Stopping" :



1 pm

Stop training at this dotted line, since dev set error $\uparrow$, after this point.

2

- Optimise cost $F^u J$
 - gradient
 - Not overfitting
 - Regularization.
- } ORTHOGONALIZATION

4

5 early stopping may backfire since it tries to solve two problems at once. (optimization & overfitting)

6

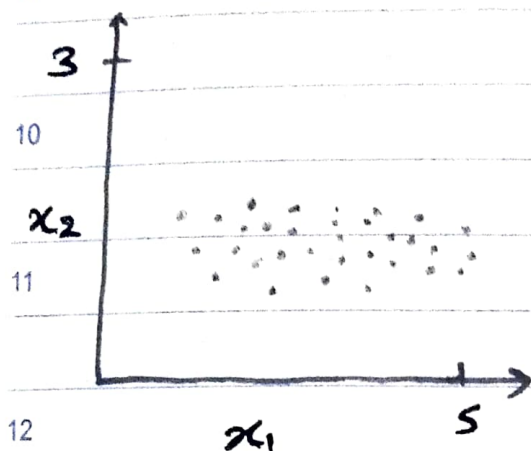
ALTERNATIVE : use L2 regularization.

7

downside : you may have to try many values for lambda.

Vid 9: Normalizing i/p s

9



Suppose you have a training set with 2 input features.

$$X = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

12

Normalizing involves 2 steps:

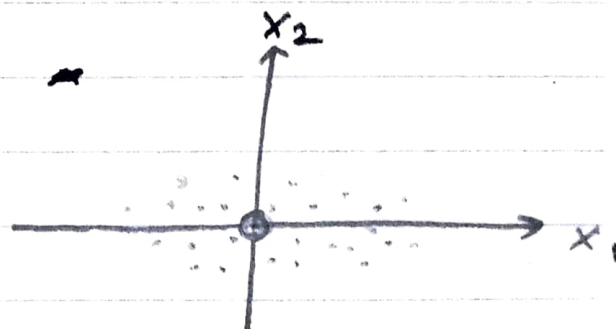
1 pm

I] Subtract ~~mean~~ mean:

2

$$\mu = \frac{1}{m} \sum_{i=1}^m X^{(i)}$$

$\Rightarrow$



3

$$X := X - \mu.$$

4 II] Normalize Variance.

5

$$\sigma^2 = \frac{1}{m} \sum_{i=1}^m [X^{(i)}]^2 ; X \neq \sigma^2$$

6

σ^2 is a vector with variances of each of the features.

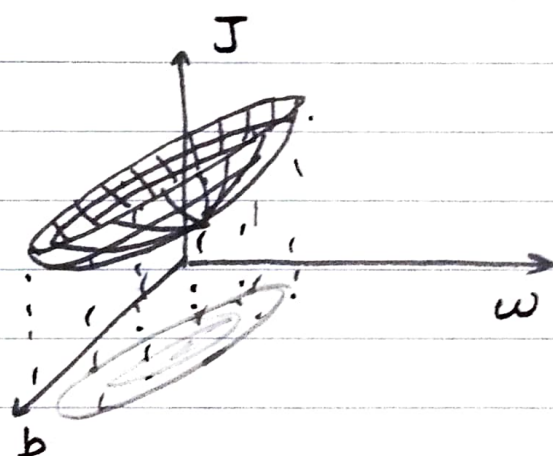
7

Why Normalize?

$$J(w, b) = \frac{1}{m} \sum_{i=1}^m \mathcal{L}(\hat{y}^{(i)}, y^{(i)})$$

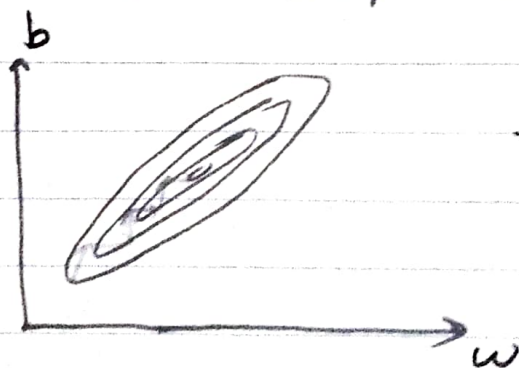
If un-normalized:

It is more likely that the cost fn will look like this:



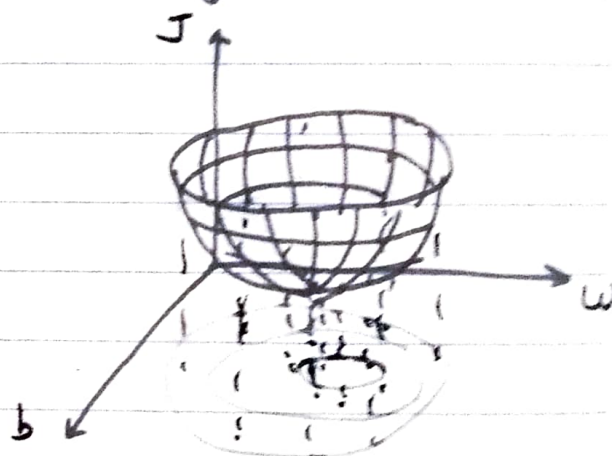
very elongated,

If contours are plotted:



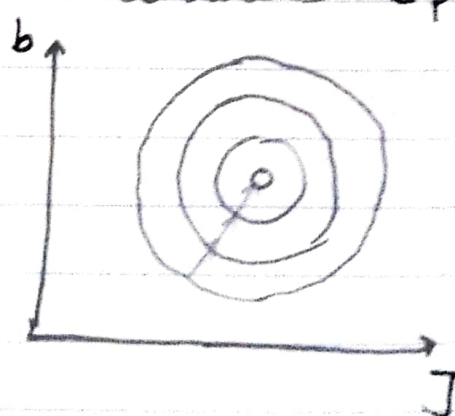
If normalized:

then the cost fn will look more symmetric



very symmetric

If contours are plotted



- very small α $\therefore$ more steps needed to reach minima

Birthday / Anniversary

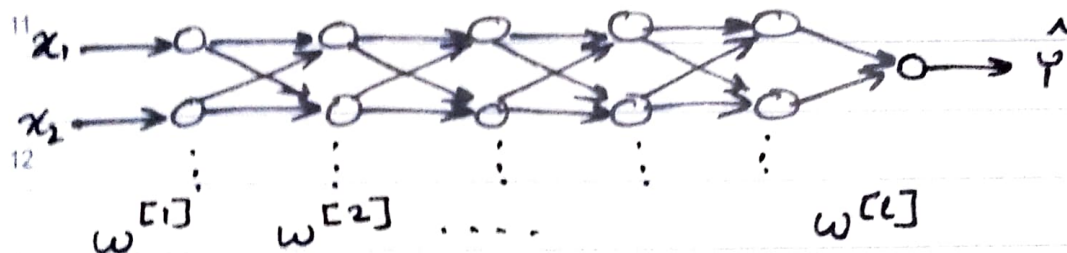
- larger α can be chosen, $\therefore$ lesser steps needed

Vid10: Vanishing or Exploding Gradients

9

The derivatives can either get very very large or very very small.

10



12

1 pm

lets take $g(z) = z$, $b^{[1]} = 0$

2

we can show that $y = w^{[L]} \cdot w^{[L-1]} \cdot w^{[L-2]} \dots w^{[2]} w^{[1]} x$

3

each $w^{[L]} = \begin{bmatrix} 1.5 & 0 \\ 0 & 1.5 \end{bmatrix}$ (say)

4

then $\hat{y} = w^{[L]} \begin{bmatrix} 1.5 & 0 \\ 0 & 1.5 \end{bmatrix}^{L-1} \cdot x$ (proof)

5

14 Sunday

6

$$\therefore \hat{y} = 1.5^L x$$

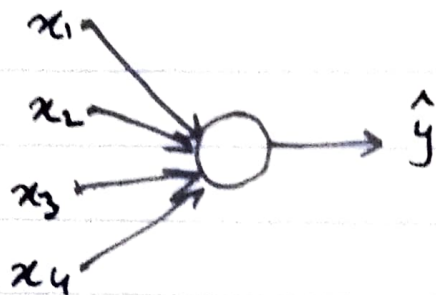
7

if there are many layers, ($L \uparrow \uparrow$)
then $\hat{y}$ will "explode" (since exponential f^L)

If 0.5 was taken, then $\hat{y}$ will "vanish"
Similarly $\hat{y} = 0.5^L x$

Vid11: Weight Initialization

eg. Single Neuron



$$Z = w_1 x_1 + w_2 x_2 + \dots + w_n x_n + b$$

$n \uparrow \uparrow$, w_i should be smaller.

We can set

$$\text{Var}(w_i) = \frac{1}{n} \left[\frac{2}{n} \text{ if ReLU}(z) \text{ is used} \right]$$

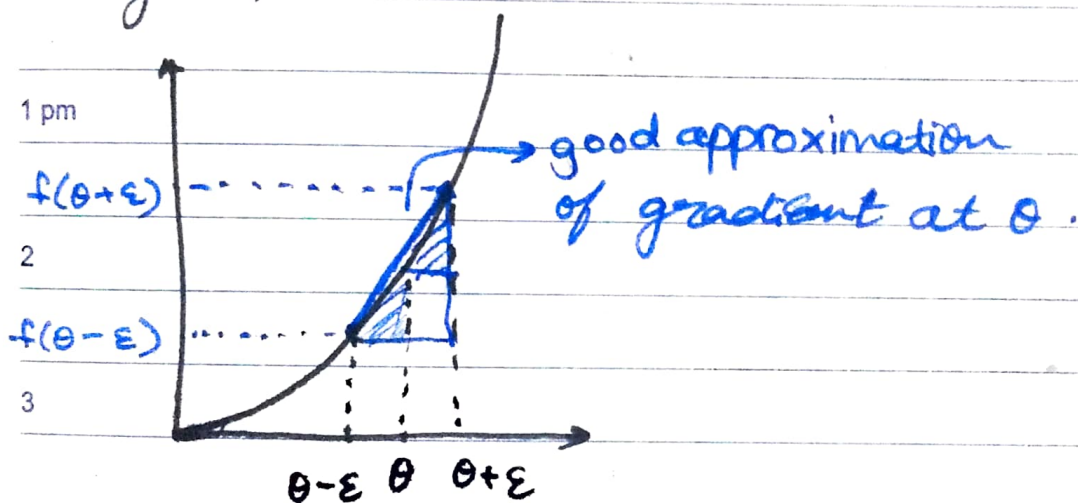
$$W^{[1]} = \text{np.random.randn}(\text{shape}..) * \sqrt{\frac{2}{n(n+1)}}$$

This causes z to be between 0 and 1
so it solves the vanishing/exploding problem

Vid 2: Numerical Approximation of gradients.

There's a method called "gradient-checking" which helps to check if we have implemented back propagation correctly.

eg. $f(\theta) = \theta^3$



$\epsilon = 0.01$

$$\frac{f(\theta + \epsilon) - f(\theta - \epsilon)}{2\epsilon} \approx \text{~~the~~} f'(\theta)$$

★ The defⁿ of derivative is

$$f'(\theta) = \lim_{\epsilon \rightarrow 0} \frac{f(\theta + \epsilon) - f(\theta - \epsilon)}{2\epsilon}.$$

Vid 13: Gradient Checking Implementation.

Take $W^{[1]}, b^{[1]}, \dots, W^{[L]}, b^{[L]}$ and reshape into a big vector θ . (using concatenation)

$$J(W^{[1]}, b^{[1]}, W^{[2]}, b^{[2]}, \dots, W^{[L]}, b^{[L]}) = J(\theta)$$

Similarly, take $dW^{[1]}, db^{[1]}, \dots, dW^{[L]}, db^{[L]}$ and reshape into a big vector $d\theta$.

1 pm

for each i :

$$d\theta_{\text{approx}}[i] = \frac{J(\theta_1, \theta_2, \dots, \theta_i + \epsilon) - J(\theta_1, \theta_2, \dots, \theta_i - \epsilon)}{2\epsilon}$$

$$\approx d\theta[i] ? \left(\frac{\partial J}{\partial \theta_i} \right)$$

we have to check if $d\theta_{\text{approx}} \approx d\theta$?

To check if they are approximately equal, implement this:

$$\frac{\|d\theta_{\text{approx}} - d\theta\|_2}{\|d\theta_{\text{approx}}\|_2 + \|d\theta\|_2} \approx 10^{-7} \text{ (great!)}$$

10^{-5} (check, still good)

10^{-3} (there is reasonable chance that there is a bug)

// use $\log_{10}()$ to check

Vid14: Tips for grad-checking implementation.

9

- grad checking should only be done in debugging, NOT in training.

10

- If algorithm fails gradcheck (value is $10^{-3} <$)

11

- then look at the components to identify bug.

12

- remember regularization while extracting ds.

1 pm

- Doesn't work with drop-out.

2

- So implement grad-check after turning off drop-out & then turn it back on.

3

- Run at random initialization

4

Vid 15: Mini-Batch gradient descent.

Vectorization:

$$X = [x^{(1)}, x^{(2)}, x^{(3)} \dots \dots \dots x^{(m)}]$$

$$Y = [y^{(1)}, y^{(2)}, y^{(3)} \dots \dots \dots y^{(m)}]$$

What if $m = 50,000,000$?

what we can do is divide the Training Set into smaller 'mini' training sets.

$$X = \underbrace{[x^{(1)}, x^{(2)} \dots x^{(1000)}]}_{\text{one mini training set}} \mid \underbrace{[x^{(1001)} \dots x^{(2000)}]}_{\text{one mini training set}} \mid \dots \mid \underbrace{x^{(m)}}_{x^{(50000)}}$$

each miniset will be denoted by $x^{\{t\}}$

$$\text{so, } \{x^{(1)} \dots x^{(1000)}\} \equiv x^{\{1\}}$$

$$\{x^{(1001)} \dots x^{(2000)}\} \equiv x^{\{2\}}$$

we will also split Y into similar minibatches

$$\text{Mini Batch } t: x^{\{t\}}, y^{\{t\}}$$

8 am

for $t = 1, \dots, 5000$

$\left\{ \begin{array}{l} \text{1 step of gradient descent} \\ \text{using } X^{\{t\}}, Y^{\{t\}} \end{array} \right\}$
 (as if $m = 5000$)

Forward prop on $X^{\{t\}}$

$$\left. \begin{array}{l} Z^{[1]} = W^{[1]} X^{\{t\}} + b^{[1]} \\ A^{[1]} = g^{[1]}(Z^{[1]}) \\ \vdots \\ A^{[L]} = g^{[L]}(Z^{[L]}) \end{array} \right\} \text{Vectorized implementation}$$

Compute cost $J = \frac{1}{1000} \sum_{i=1}^L \mathcal{L}(\hat{y}^{(i)}, y^{(i)})$

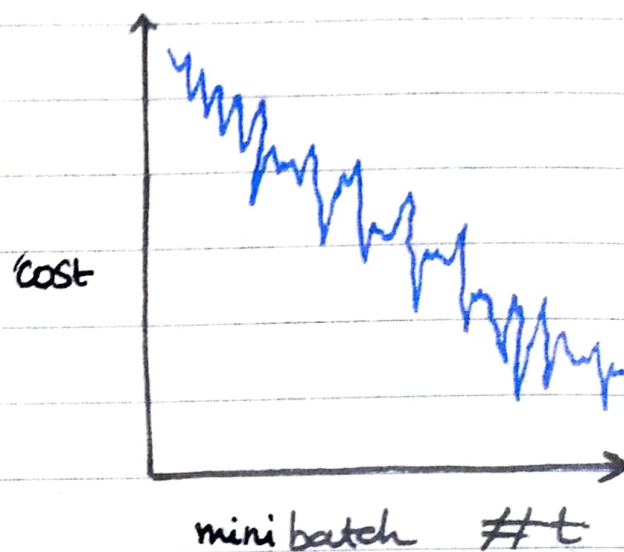
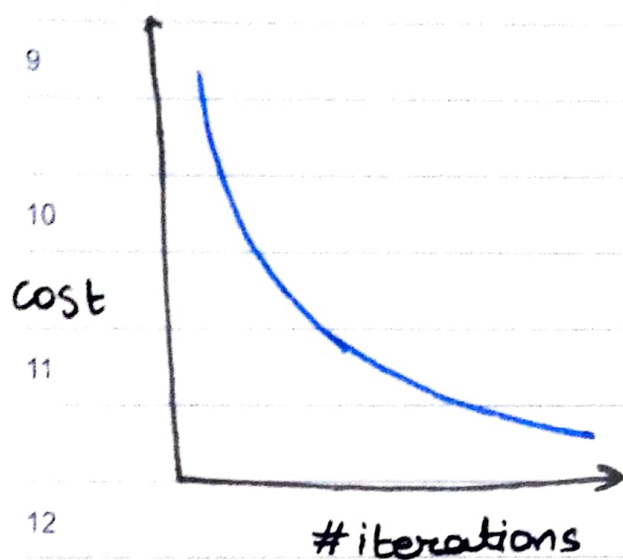
$$+ \frac{\lambda}{2(1000)} \sum_l \|W^{[l]}\|_F^2$$

Backprop to compute gradients w.r.t $J^{\{t\}}$ (using $(X^{\{t\}}, Y^{\{t\}})$)

$$W^{[l]} := W^{[l]} - \alpha dW^{[l]}, \quad b^{[l]} := b^{[l]} - \alpha db^{[l]}$$

21 Sunday

8 am Vid 16: Understanding Mini-Batch Gradient descent.



1 pm [Batch gradient descent]

[mini batch gradient descent]

2 Plot $J^{\{t\}}$ using $X^{\{t\}}, Y^{\{t\}}$

- 3
- Choosing size of minibatch.
- 4 1) If minibatch size = m , we just get Batch gradient descent.
- 5 2) If $\text{batch size} = 1$, we get "Stochastic gradient descent".
- 6 Every example is a mini-batch.
- 7 Let's see what 1) & 2) will do on optimizing this cost function.

8 am

9

10

11

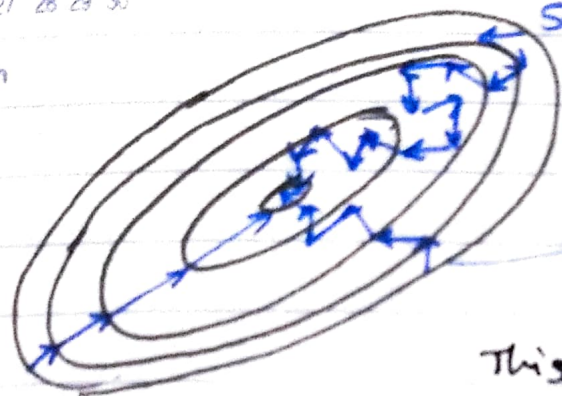
12

1 pm

2

3

4



Size = 1

Size = m

Ideally we should select something in between 1 and m.

↓
This gives Fastest learning.

How to choose optimum size?

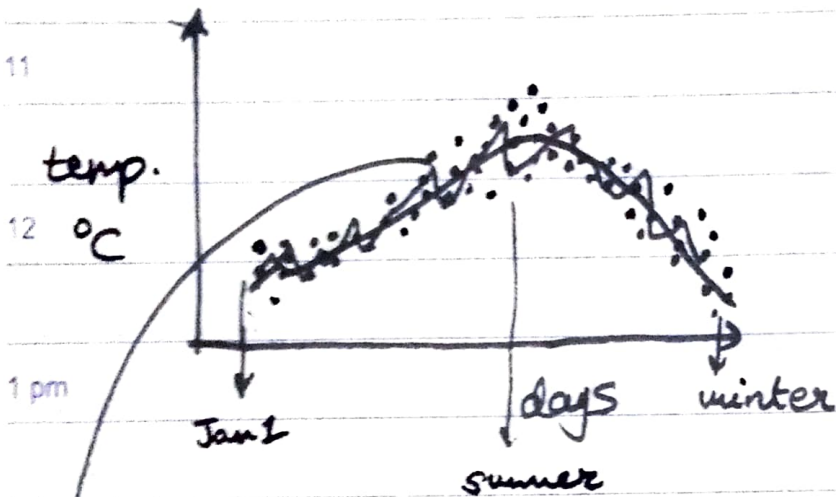
- If you have a small training set, take batch size as m, no point in doing mini-batch gradient desc.

$m \approx 2000$

- try powers of 2, (64, 256, 1024) ... as batch sizes.

Vid17: Exponentially weighted Averages.

- Faster than gradient descent.



$$V_0 = 0$$

$$V_1 = 0.9 V_0 + 0.1 \theta_1$$

$$V_2 = 0.9 V_1 + 0.1 \theta_3$$

⋮

$$V_t = 0.9 V_{t-1} + 0.1 \theta_t$$

$$V_t = \beta V_{t-1} + (1-\beta) \theta_t$$

V_t is approximately averaging over $\frac{1}{1-\beta}$ days temp.

$$\rightarrow \beta = 0.9$$

smoother curve for $\beta = 0.98$

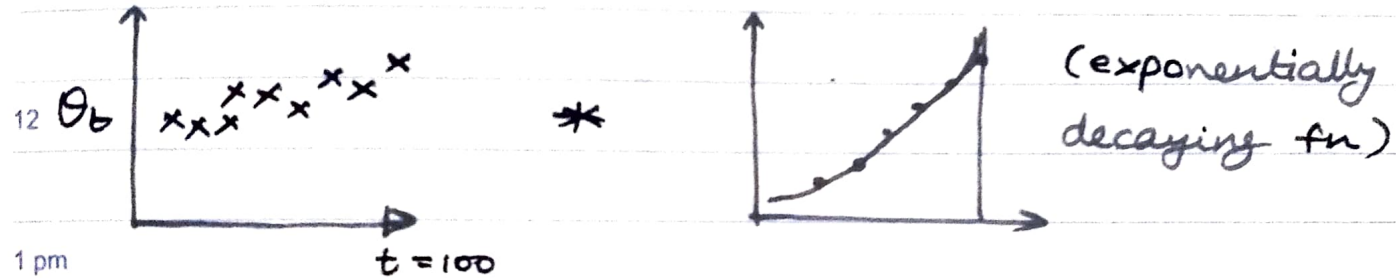
8 am Vid 18: Understanding exponentially weighted avg's.

9 $V_{100} = 0.9 V_{99} + 0.1 \theta_{100}$

$$= 0.1 \theta_{100} + 0.9 (0.1 \theta_{99} + 0.9 V_{98})$$

10 $= 0.1 \theta_{100} + 0.1 \times 0.9 \theta_{99} + 0.1 (0.9)^2 \theta_{98} + 0.1 (0.9)^3 \theta_{97} \dots$

11



2 $V_t = \beta V_t + (1 - \beta) \theta_t$

3 Vid 19: Bias correction

4 $\frac{V_t}{1 - \beta^t}$

5

6 $t = 2: 1 - \beta^t = 1 - (0.98)^2 = 0.0396$

7

$$\frac{V_2}{1 - \beta^2} = \frac{0.0196 \theta_1 + 0.02 \theta_2}{0.0396}$$

[notice that $0.0196 + 0.02 = 0.0396$]
 $\therefore$ It becomes weighted average of θ_1 & θ_2 .

Vid20. Gradient Descent with momentum.

Gradient descent ~~with momentum~~:



↕ slow learning
↔ fast learning.

with momentum:

On iteration t :

compute dw , db on current minibatch.

$$V_{dw} = \beta V_{dw} + (1-\beta) dw$$

$$V_{db} = \beta V_{db} + (1-\beta) db$$

$$w := w - \alpha V_{dw}$$

$$b := b - \alpha V_{db}$$

This smooths out the oscillation

Instead of  $\Rightarrow$ 

ALGO:

$$v_{dw} = \beta v_{dw} + (1-\beta) dw$$

$$v_{db} = \beta v_{db} + (1-\beta) db$$

$$W = W - \alpha v_{dw}, \quad b = b - \alpha v_{db}$$

β is usually 0.9.

Vid 21. RMSProp

(Root Mean Square Prop)

1 pm

On iteration t :

Compute dw , db on current mini-batch.

$$S_{dw} = \beta S_{dw} + (1-\beta) dw^2 \quad // \text{should be small}$$

$$S_{db} = \beta S_{db} + (1-\beta) db^2 \quad // \text{should be large}$$

$$w := w - \frac{\alpha dw}{\sqrt{S_{dw} + \epsilon}} \quad b := b - \frac{\alpha db}{\sqrt{S_{db} + \epsilon}}$$

28 Sunday

(So that $\frac{1}{0}$ doesn't happen)

Vid 22: Adam Optimization Algorithms

$$V_{dw} = 0 \quad S_{dw} = 0 \quad V_{db} = 0 \quad S_{db} = 0$$

At iteration t :

Compute dw , db using current minibatch.

$$\begin{aligned} V_{dw} &= \beta_1 V_{dw} + (1 - \beta_1) dw \\ V_{db} &= \beta_1 V_{db} + (1 - \beta_1) db \end{aligned} \quad \left. \vphantom{\begin{aligned} V_{dw} &= \beta_1 V_{dw} + (1 - \beta_1) dw \\ V_{db} &= \beta_1 V_{db} + (1 - \beta_1) db \end{aligned}} \right\} \text{"momentum"} (\beta_1)$$

$$\begin{aligned} S_{dw} &= \beta_2 S_{dw} + (1 - \beta_2) dw^2 \\ S_{db} &= \beta_2 S_{db} + (1 - \beta_2) db^2 \end{aligned} \quad \left. \vphantom{\begin{aligned} S_{dw} &= \beta_2 S_{dw} + (1 - \beta_2) dw^2 \\ S_{db} &= \beta_2 S_{db} + (1 - \beta_2) db^2 \end{aligned}} \right\} \text{"RMSprop"} (\beta_2)$$

$$\begin{aligned} V_{dw}^{\text{corrected}} &= V_{dw} / (1 - \beta_1^t) \\ V_{db}^{\text{corrected}} &= V_{db} / (1 - \beta_1^t) \end{aligned}$$

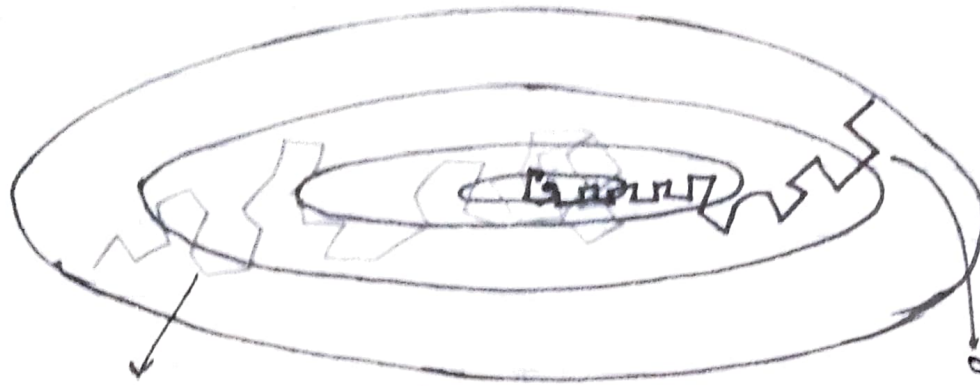
$$\begin{aligned} S_{dw}^{\text{corrected}} &= S_{dw} / (1 - \beta_2^t) \\ S_{db}^{\text{corrected}} &= S_{db} / (1 - \beta_2^t) \end{aligned}$$

$$\begin{aligned} w &:= w - \alpha \frac{V_{dw}^{\text{corrected}}}{\sqrt{S_{dw}^{\text{corrected}} + \epsilon}} & b &:= b - \alpha \frac{V_{db}^{\text{corrected}}}{\sqrt{S_{db}^{\text{corrected}} + \epsilon}} \end{aligned}$$

Hyperparameters choice:

$$\begin{aligned} \alpha &: \text{needs to tune} & \beta_1 &: 0.9 \\ \beta_2 &: 0.999 & \epsilon &: 10^{-8} \end{aligned}$$

Vid23: Learning rate decay.



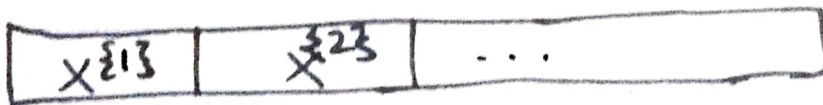
α is not reduced

α is reduced slowly

The logic is that if you are far away from min, you can afford to take larger steps, but as you go closer, your steps are reduced so that even if you wander, you are still close to the minimum.

Implementation:

1 epoch = 1 pass through data



1st epoch
2nd epoch

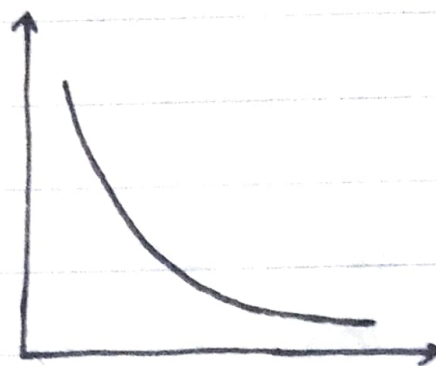
$$\alpha = \frac{1}{1 + \gamma * (\#epoch)} \alpha_0$$



decay rate (hyperparameter)

10

epoch	α (for $\alpha_0 = 0.2, \gamma = 1$)
1	0.1
2	0.67
3	0.5
4	0.4
⋮	⋮



1 pm

2

OTHER LEARNING RATE DECAYS:

3

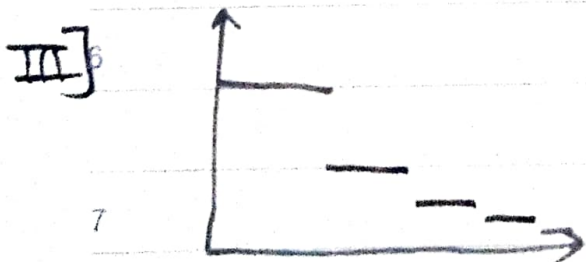
#epoch

$$\text{I)} \quad \alpha = (0.95)^{\#epoch} \cdot \alpha_0$$

4

$$\text{II)} \quad \alpha = \frac{k}{\sqrt{\#epoch}} \cdot \alpha_0 \quad \text{or} \quad \frac{k}{\sqrt{t}} \alpha_0$$

5



7

Vid 24: Tuning Process

9

Hyperparameters :

10

α (most imp)

β

11

$\beta_1, \beta_2, \epsilon$

layers

12

hidden units

γ

1 pm

mini-batch size

2

dont use grid, use random points.

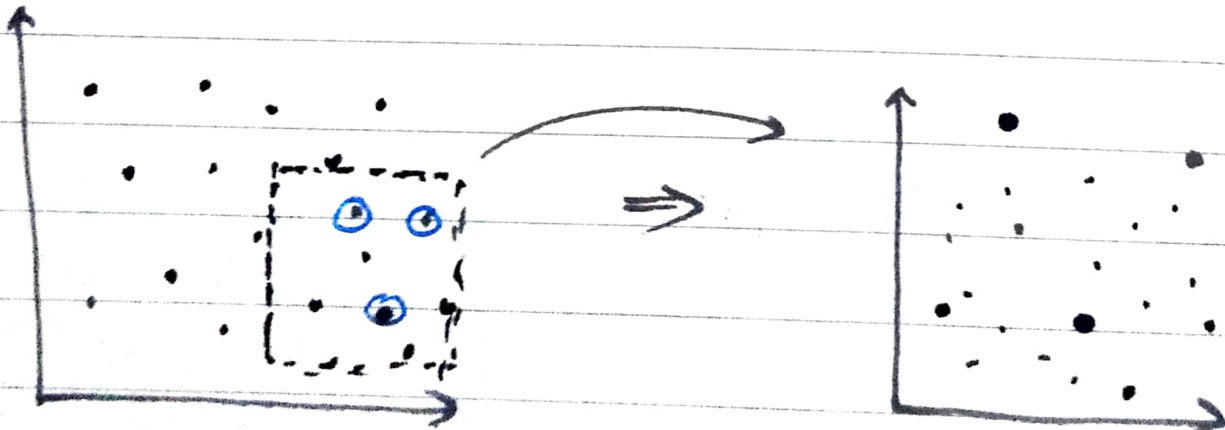
3

~~Vid 25~~: Coarse to fine

4

5

6



Vid25: using Appropriate Scale.

$$n^{[1]} = 50 \dots 100$$

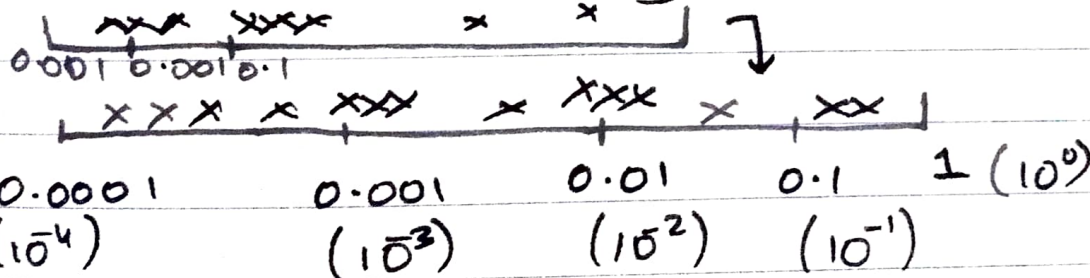


#layers $L : 2 - 4$ 2, 3, 4

This won't work for α .

Since α is between 0 & 1 (and very small)

So convert to log scale.

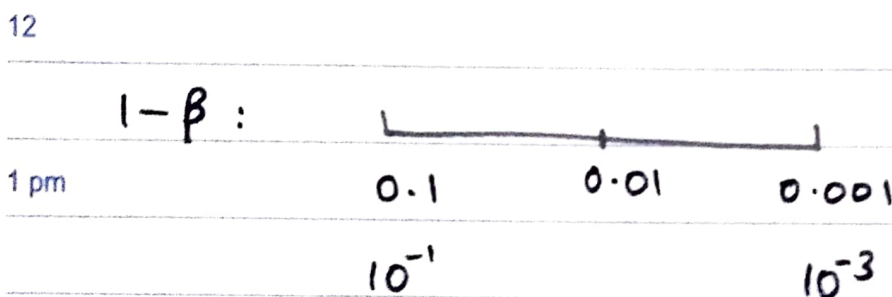


$$r = -4 * \text{np.random.rand}() \leftarrow \sigma \in [-4, 0]$$

$$\alpha = 10^r$$

Hyperparameters for exponentially weighted averages.

$$\beta = 0.9 \dots\dots 0.999$$



$$r \in [-3, -1]$$

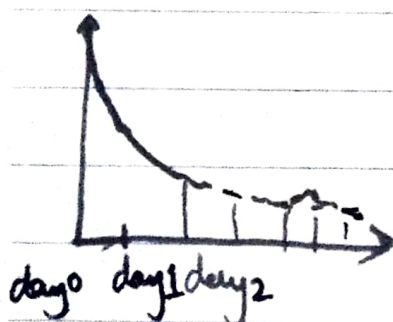
$$1 - \beta = 10^r$$

$$\beta = 1 - 10^r$$

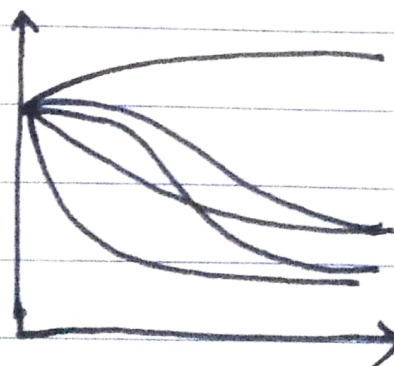
Vid 26: Hyperparameter tuning in practice.

Babysit 1 model.

04 Sunday



Train many models in parallel



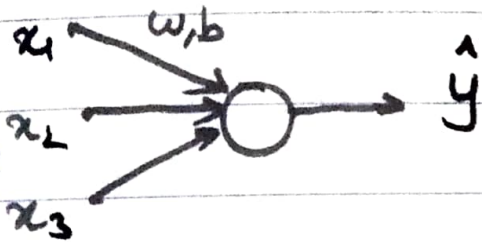
(will have to do this)
(less computational power)



Vid 27: Normalizing Activations in a network.

→ makes hyperparameter search faster.

NORMALIZING $1/p^s$ TO SPEED UP LEARNING.



$$\mu = \frac{1}{m} \sum_i x^{(i)}$$

$$X = X - \mu$$

$$\sigma^2 = \frac{1}{m} \sum_i (x^{(i)})^2 \text{ (element wise)}$$

$$X = X / \sigma^2$$

can we normalize $a^{[2]}$ so as to train $w^{[3]}, b^{[3]}$ faster.

Normalize $z^{[2]}$

IMPLEMENTING BATCH NORM.

9

given some intermediate values in NN

10 $z^{(1)} \dots z^{(m)}$

11
$$\mu = \frac{1}{m} \sum_i z^{(i)}$$

12
$$\sigma^2 = \frac{1}{m} \sum_i (z^{(i)} - \mu)^2$$

1 pm

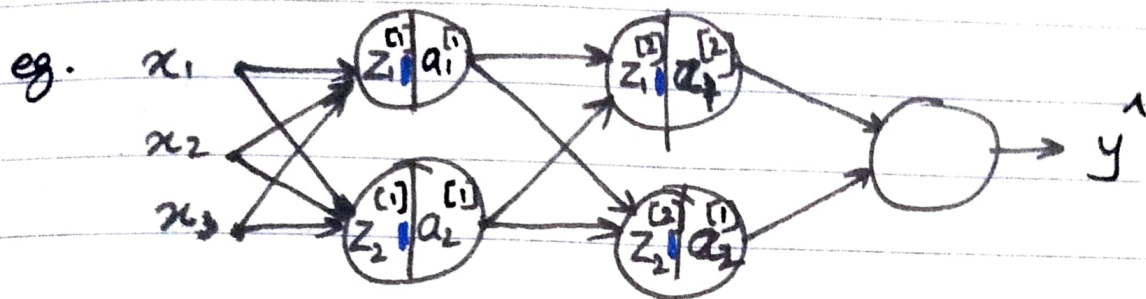
2
$$z_{\text{norm}}^{(i)} = \frac{z^{(i)} - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

3
$$\tilde{z}^{(i)} = \gamma_L z_{\text{norm}}^{(i)} + \beta_L$$

4 $\gamma_L \in \mathbb{R}$ & $\beta_L \Rightarrow$ learnable parameters of model.

if $\gamma_L = \sqrt{\sigma^2 + \epsilon}$ $\beta_L = \mu$

Vid 28: Fitting Batch Norm into NN's



$$X \xrightarrow{W^{[1]}, b^{[1]}} Z^{[1]} \xrightarrow{\text{Batch Normalization } (\beta_N^{[1]}, \gamma_N^{[1]})} \tilde{Z}^{[1]} \xrightarrow{\downarrow} a^{[1]} = g^{[1]}(\tilde{Z}^{[1]})$$

Parameters: $W^{[1]}, b^{[1]}, W^{[2]}, b^{[2]}, \dots, W^{[L]}, b^{[L]}$
 $\beta_N^{[1]}, \gamma_N^{[1]}, \beta_N^{[2]}, \gamma_N^{[2]}, \dots, \beta_N^{[L]}, \gamma_N^{[L]}$

Batch normalization can be implemented using single line of code

WORKING w MINI Batches

$$X^{\{1\}} \xrightarrow{W^{[1]}, b^{[1]}} Z^{[1]} \xrightarrow{\beta_N^{[1]}, \gamma_N^{[1]} \text{ BN}} \tilde{Z}^{[1]} \rightarrow g^{[1]}(\tilde{Z}^{[1]}) = a^{[1]} \dots$$

$$X^{\{2\}} \xrightarrow{W^{[1]}, b^{[1]}} Z^{[1]} \xrightarrow{\beta_N^{[1]}, \gamma_N^{[1]} \text{ BN}} \tilde{Z}^{[1]} \rightarrow \dots$$

$$z^{[1]} = w^{[1]} a^{[1-1]} + b^{[1]}$$

9

10 If batch normalization is used, you can omit the ' $b^{[1]}$ ' parameter.

11

$$z^{[1]} = w^{[1]} a^{[1-1]} + \beta_L^{[1]}$$

12

use back prop to compute:
1 pm $dw^{[1]}, d\beta_N^{[1]}, d\gamma_N^{[1]}$

update:

$$\begin{aligned} w^{[1]} &:= w^{[1]} - \alpha dw^{[1]} \\ \beta_N^{[1]} &:= \beta_N^{[1]} - \alpha d\beta_N^{[1]} \\ \gamma_N^{[1]} &:= \gamma_N^{[1]} - \alpha d\gamma_N^{[1]} \end{aligned}$$

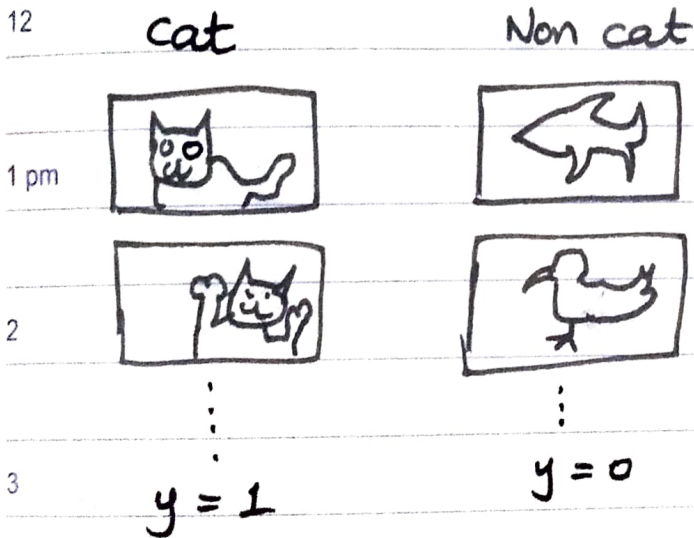
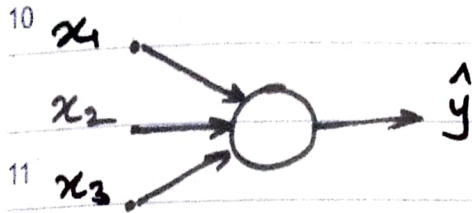
3

4 Works with momentum, RMS Prop, Adam.

5

Vid 29: Why does batch Norm work?

LEARNING ON SHIFTING I/P DISTRIBUTION.



Suppose only black cats are used to train.

If this model is used on multicolored cats, not just black cats, then NN might not perform well.

This shift of data is called "Covariate Shift".

$$X \longrightarrow Y$$

Why is Covariate Shift a problem with Neural networks?

x affects $a_1^{[k]}, a_2^{[k]}, a_3^{[k]} \dots$
which further affect $z_1^{[k+1]}, z_2^{[k+1]}, z_3^{[k+1]}$
...

Vid 30: Batch Norm at test time.

equations used for batch normalization

$$\mu = \frac{1}{m} \sum_i z^{(i)}$$

$$\sigma^2 = \frac{1}{m} \sum_i (z^{(i)} - \mu)^2$$

$$z_{\text{norm}}^{(i)} = \frac{z^{(i)} - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

→ estimate μ & σ^2 using exponentially weighted average. (across mini-batches)

~~$x^{[1]}$~~ , ~~$x^{[2]}$~~ , ...

11 Sunday

$$\tilde{z}^{(i)} = \gamma_N z_{\text{norm}}^{(i)} + \beta_N$$

Implementation on next page.

8 am

 μ, σ^2 $x^{\{1\}}, x^{\{2\}}, x^{\{3\}}, \dots$ $\mu^{\{1\}}$ $\mu^{\{1\}[1]}$ θ_1 $\sigma^2[1]\{1\}$ $\mu^{\{2\}}$ $\mu^{\{2\}[1]}$ θ_2 $\sigma^2\{2\}[1]$ $\mu^{\{3\}}$ $\mu^{\{3\}[1]}, \dots$ θ_3 $\sigma^2\{3\}[1]$ $\Rightarrow$ μ_{exp} $\Rightarrow$ σ_{exp}^2

$$Z_{norm} = \frac{x - \mu_{exp}}{\sqrt{\sigma_{exp}^2 + \epsilon}}$$

$$\tilde{z} = \gamma_N Z_{norm} + \beta_N$$

1 pm

2

3

4

Vid 31: Softmax Regression.

9

10 eg. Instead of recognizing just cats, you want to recognize cats, dogs & baby chicks.

11 cat $\rightarrow 1$, dog $\rightarrow 2$, chick $\rightarrow 3$
else $\rightarrow 0$.

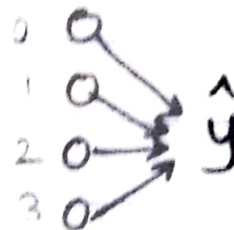
12

$C \rightarrow$ denotes no. of classes (here $C = 0, 1, 2, 3$)
1 pm # classes = 4.

2

So $n^{[L]} = C$.

i.e.



3

$\hat{y}$ will be a $(4, 1)$ dimensional vector.
(in general $(C, 1)$)

4

$$z^{[L]} = w^{[L]} a^{[L-1]} + b^{[L]}$$

5

6 Here a weird activation function is used (softmax f^n)

7

$$a_i^{[L]} = \frac{e^{z_i^{[L]}}}{\sum_{j=1}^C t_j}$$

i.e. $a_i^{[L]} = \frac{t_i}{\sum_{i=1}^C b_i}$

8 am eg.

9 $z^{[L]} = \begin{bmatrix} 5 \\ 2 \\ -1 \\ 3 \end{bmatrix}$

10 $t = \begin{bmatrix} e^5 \\ e^2 \\ e^{-1} \\ e^3 \end{bmatrix} = \begin{bmatrix} 148.4 \\ 7.4 \\ 0.4 \\ 20.1 \end{bmatrix}$

12 So summing up elements

1 pm $\sum_{j=1}^4 t_j = e^5 + e^2 + e^{-1} + e^3 = 176.3$

2 now $a^{[L]} = \frac{t}{176.3}$

3 $\therefore \begin{array}{c} \text{O} \rightarrow \end{array} \frac{e^5}{176.3} = 0.842 \quad P(\text{class 0}) = 84.2\%$

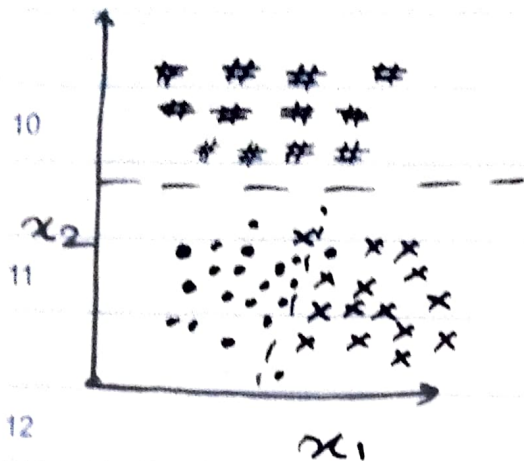
4 $\begin{array}{c} \text{O} \rightarrow \end{array} \frac{e^2}{176.3} = 0.042 \quad P(\text{class 1}) = 4.2\%$

5 $\begin{array}{c} \text{O} \rightarrow \end{array} \frac{e^{-1}}{176.3} = 0.00208 \quad P(\text{class 3}) = 0.2\%$

6 $\begin{array}{c} \text{O} \rightarrow \end{array} \frac{e^3}{176.3} = 0.1139 \quad P(\text{class 4}) = 11.39\%$

SOFTMAX EXAMPLES

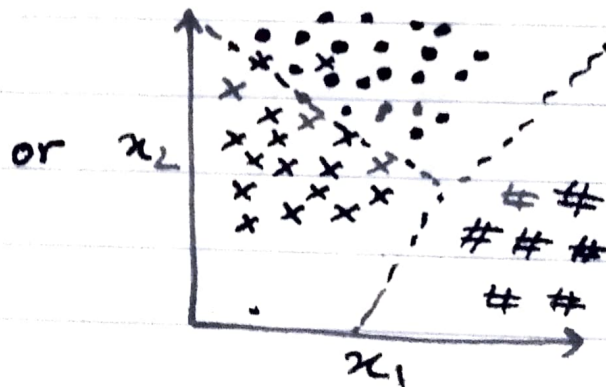
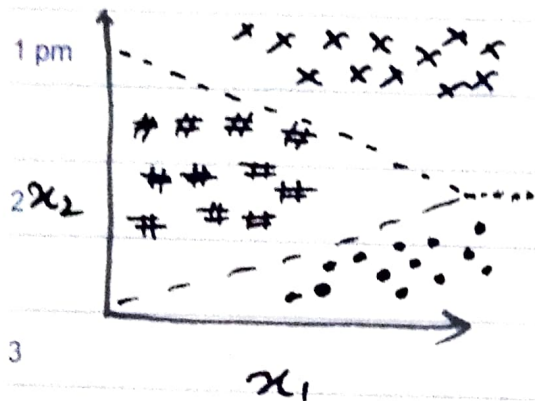
9



$$C = 3$$

separates data into 3 parts.
(classes)

12



3

4

can be done with more C .

5

Vid 32: Training a Softmax Layer.

Recall: $z^{[L]} = \begin{bmatrix} 5 \\ 2 \\ -1 \\ 3 \end{bmatrix}$ $t = \begin{bmatrix} e^5 \\ e^2 \\ e^{-1} \\ e^3 \end{bmatrix}$

$$a^{[L]} = g^{[L]}(z^{[L]}) = \begin{bmatrix} \frac{e^5}{e^5 + e^2 + e^{-1} + e^3} \\ \frac{e^2}{e^5 + e^2 + e^{-1} + e^3} \\ \frac{e^{-1}}{e^5 + e^2 + e^{-1} + e^3} \\ \frac{e^3}{e^5 + e^2 + e^{-1} + e^3} \end{bmatrix} = \begin{bmatrix} 0.842 \\ 0.042 \\ 0.002 \\ 0.114 \end{bmatrix}$$

If $C=2$, then softmax regression reduces to linear regression.

Loss Function

$$y = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix} - \text{cat}$$

$$\hat{y} = \begin{bmatrix} 0.3 \\ 0.2 \\ 0.1 \\ 0.4 \end{bmatrix}$$

$$\mathcal{L}(\hat{y}, y) = - \sum_{j=1}^4 y_j \log \hat{y}_j$$

$$y_1, y_3, y_4 = 0$$

$$\therefore \mathcal{L} = -y_2 \log \hat{y}_2 \quad \text{by } y_2 = 1,$$

$$\mathcal{L} = -\log(\hat{y}_2)$$

so, if $\mathcal{L} \downarrow$, $-\log(\hat{y}_2) \downarrow$, $\hat{y}_2 \uparrow$

$$J(w^{[1]}, b^{[1]}, \dots) = \frac{1}{m} \sum_{i=1}^m \mathcal{L}(\hat{y}^{(i)}, y^{(i)})$$

9

$$Y = [y^{(1)}, y^{(2)} \dots y^{(m)}] \quad \hat{Y} = [\hat{y}^{(1)} \dots \hat{y}^{(m)}]$$

10

$$= \begin{bmatrix} 0 & 0 & 1 & & \\ 1 & 0 & 0 & \dots & \\ 0 & 1 & 0 & & \\ 0 & 0 & 0 & & \end{bmatrix} = \begin{bmatrix} 0.1 & 0.4 & 0.6 & & \\ 0.5 & 0.1 & 0.1 & \dots & \\ 0.3 & 0.3 & 0.1 & & \\ 0.1 & 0.2 & 0.1 & & \end{bmatrix}$$

11

12

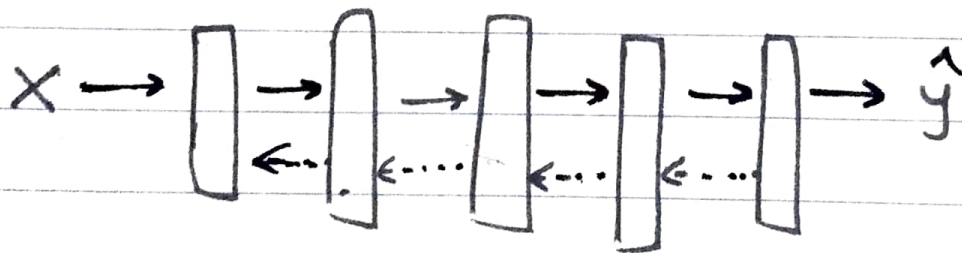
1 pm

$$\therefore \dim(Y) = (C \times m)$$

$$\dim(\hat{Y}) = (C \times m)$$

2 GRADIENT DESCENT WITH SOFTMAX :

3

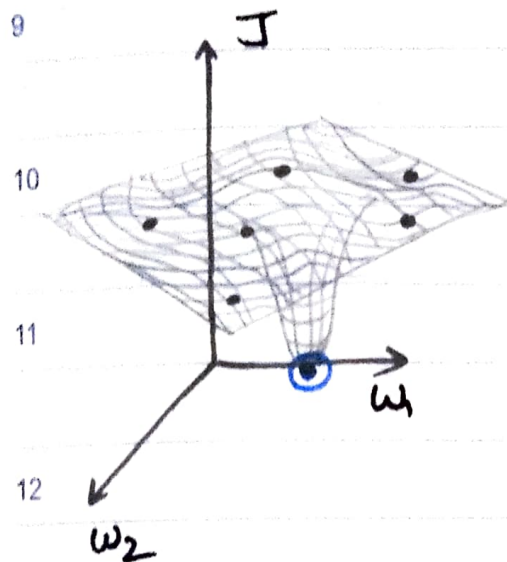


4

$$\text{Backprop: } dz^{[L]} = \hat{y} - y$$

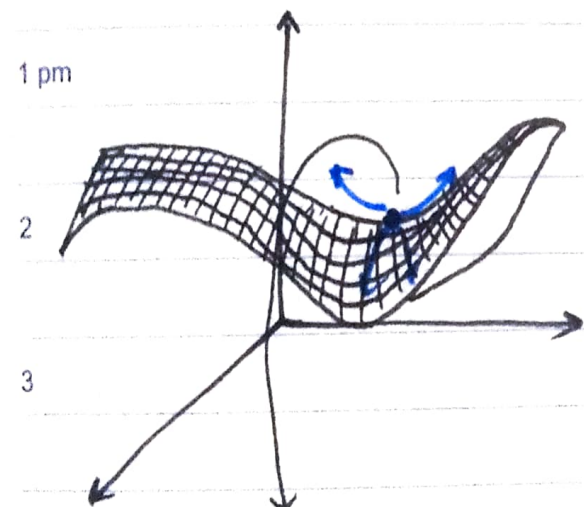
5

Vid33: Problem of Local Optima.



all these points have 0 gradient, but they are saddle points, not local optima.

⊙ → LOCAL MINIMA



The point must be concave on all sides.



This point is a saddle point. Its gradient = 0. but it is decreasing in some directions.

- It is unlikely to get stuck in a bad local optima.

Vid 34. TENSORFLOW.

9

• MOTIVATING PROBLEM

10

lets say we have a cost fⁿ J that we want to minimize.

11

$$J(w) = w^2 - 10w + 25 \Rightarrow (w-5)^2$$

12

1 pm

Code :

```
import numpy as np
```

2

```
import tensorflow as tf
```

3

```
w = tf.Variable(0, dtype=tf.float32)
```

```
# cost = √(tf.add(w**2, tf.multiply(-10, w)), 25)
```

4

```
tf.add. cost = w**2 + -10*w + 25
```

```
train = tf.train.GradientDescentOptimizer(0.01).  
minimize(cost)
```

5

```
init = tf.global_variables_initializer()
```

```
session = tf.Session()
```

7

```
session.run(init)
```

```
print(session.run(w))
```



o/p: 0



session.run(train)

9 print(session.run(w))

op: ~~0.1~~ 0.1

10

running 1000 iterations of training.

11

for i in range(1000):

12

session.run(train)

print(session.run(w))

1 pm

o/p: 4.99999

2

for input: $x = \text{tf.placeholder(tf.float32, [3, 1])}$

3

we can replace our cost with:

4

$$\text{cost} = x[0][0] * w ** 2 + x[1][0] * w + x[2][0]$$

5

coefficients = np.array([[1.], [-10.], [25.]])

6

session.run(train, feed_dict = { x: coefficients })

7